

ECOFLEET AI: INTELLIGENT ELECTRIC VEHICLE FLEET MONITORING AND ROUTE OPTIMIZATION PLATFORM

Submitted in the partial fulfilment for the Course of

CSA-1016 SOFTWARE ENGINEERING

to the award of the degree of

BACHELOR OF ENGINEERING

IN

CSE WITH AI

Submitted by

KENISHA JUDITH J (192572071)

Under the Supervision of

ANITADAVAMANI K



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

1. Problem Overview

1.1 Introduction

The transportation and logistics industry is undergoing a major transformation with the increasing adoption of electric vehicles (EVs). Governments and organizations across the world are promoting sustainable transportation to reduce carbon emissions, decrease dependence on fossil fuels, and improve environmental performance. As a result, electric vehicle fleets are being widely used in logistics, delivery services, public transportation, and corporate mobility solutions.

Managing an electric vehicle fleet is significantly more complex than managing conventional fuel-based vehicles. Fleet operators must continuously monitor battery health, charging schedules, energy consumption, vehicle availability, and route efficiency. Any inefficiency in these areas can lead to increased operational costs, reduced battery life, delivery delays, and lower fleet utilization.

Recent advancements in Artificial Intelligence (AI), Internet of Things (IoT), GPS technologies, and cloud computing have enabled the development of intelligent fleet management systems. These technologies can collect real-time data from vehicles, analyze operational patterns, predict maintenance requirements, and recommend optimized routes. By combining real-time monitoring with predictive analytics, organizations can improve decision-making and enhance overall fleet performance.

EcoFleet AI is developed as an intelligent electric vehicle fleet monitoring and route optimization platform that addresses these challenges through a data-driven and scalable software solution.

1.2 Background of the Problem

Traditional fleet management systems mainly focus on vehicle tracking and basic reporting. Although these systems provide information about vehicle location and trip history, they often lack intelligent capabilities such as battery degradation prediction, charging optimization, and energy-aware route planning.

In electric vehicle operations, battery performance is one of the most critical factors affecting productivity. Frequent fast charging, poor charging schedules, and excessive energy consumption can accelerate battery degradation and increase maintenance costs. Similarly, inefficient route planning may result in unnecessary charging stops, longer travel times, and higher operational expenses.

Fleet managers also require real-time visibility into vehicle health, charging station availability, and energy usage patterns. Without such visibility, it becomes difficult to make informed operational decisions. Therefore, there is a need for a smart platform that can continuously monitor fleet performance, analyze battery behavior, optimize charging operations, and recommend efficient routes.

1.3 Problem Statement

Electric vehicle fleet operators face multiple operational challenges, including battery degradation, inefficient charging schedules, increased energy consumption, route delays, and limited visibility into vehicle health and charging infrastructure. Existing fleet management solutions primarily provide tracking functionality but do not support intelligent decision-making or predictive analytics.

Poor route planning can lead to unnecessary charging stops, increased operational costs, and delayed deliveries. Similarly, the absence of predictive maintenance may result in unexpected vehicle downtime and reduced fleet availability. Fleet managers require a system that can integrate real-time monitoring, analytics, and optimization capabilities into a single platform.

Hence, an AI-powered fleet monitoring and route optimization system is required to improve operational efficiency, reduce energy consumption, enhance battery life, and support sustainable transportation objectives.

1.4 Proposed Solution

The proposed solution, EcoFleet AI, is a web-based intelligent fleet management platform designed to support electric vehicle operations. The system integrates real-time monitoring, analytics, and optimization features to provide comprehensive operational visibility.

The platform monitors vehicle status, battery charge level, energy consumption, and route information. It also provides analytical dashboards that help fleet managers understand fleet utilization and operational performance. By using intelligent route recommendations and charging support mechanisms, the system aims to reduce energy wastage and improve fleet productivity.

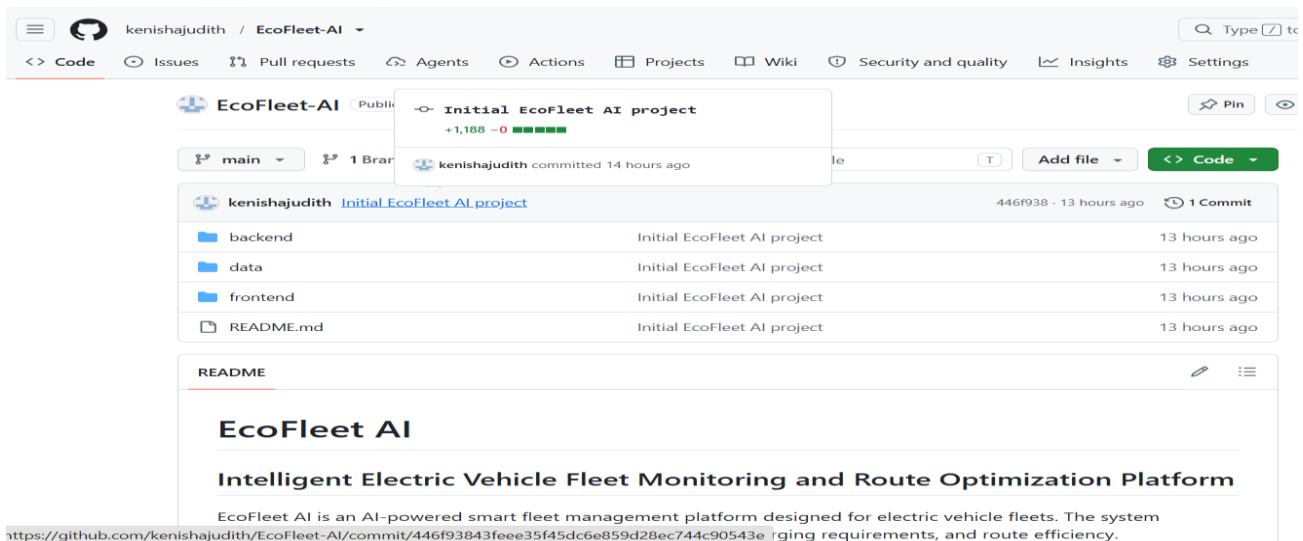


Figure 1: EcoFleet AI GitHub Repository Home Page

Major Features of the System

- Real-time vehicle monitoring
- Battery health and charge tracking
- Energy consumption analysis
- Route optimization support
- Charging management support
- Fleet analytics and reporting dashboard

1.5 Project Objectives

The primary objectives of EcoFleet AI are:

- Continuously monitor vehicle status and operational parameters.
- Track battery health and energy consumption in real time.
- Improve route efficiency through intelligent recommendations.
- Support better charging management decisions.
- Provide analytical dashboards for fleet managers.
- Reduce operational costs and energy usage.
- Improve fleet utilization and vehicle availability.

1.6 Scope of the Project

The scope of the project includes the development of a full-stack fleet management application for electric vehicle operations. The system focuses on monitoring, analytics, and optimization functionalities. It includes user interface components, backend processing, data visualization, and operational reporting.

The project does not include integration with physical IoT hardware, real charging station networks, or commercial fleet management infrastructure. However, the architecture is designed in a modular manner so that such integrations can be added in future enhancements.

1.7 Expected Benefits

Implementation of EcoFleet AI can provide several operational and technical benefits:

- Improved battery life through better charging practices.
- Enhanced fleet utilization.
- Reduced energy consumption.
- Lower operational and maintenance costs.
- Better decision-making using analytics.

- Improved route efficiency.
- Support for sustainable transportation initiatives.

1.8 Conclusion of Problem Overview

EcoFleet AI addresses an important real-world problem in the field of sustainable transportation and intelligent fleet management. The project combines real-time monitoring, analytics, and optimization to improve the efficiency of electric vehicle fleet operations. By integrating vehicle monitoring, battery analysis, route optimization, and operational dashboards into a single platform, the system provides a strong foundation for future development of smart and scalable fleet management solutions.

This problem overview establishes the context for the subsequent evaluation of repository organization, code quality, version control practices, testing, documentation, and overall maintainability of the EcoFleet AI project.

2. Repository Organization

2.1 Repository Structure

The EcoFleet AI repository is organized using a modular folder structure that separates the major components of the application. This organization improves maintainability, readability, and collaboration among developers. The repository contains dedicated folders for backend logic, frontend resources, datasets, and project documentation.

The current repository structure is as follows:

EcoFleet-AI/

├── backend/ ├── data/ ├── frontend/ └── README.md

This structure follows the principle of separation of concerns, where each folder is responsible for a specific part of the application.

2.2 Folder Analysis

Backend Folder

The backend folder contains the server-side implementation of the application. It includes business logic, API handling, data processing, and fleet management functionalities. Separating backend logic from frontend components improves scalability and simplifies debugging and testing.

Data Folder

The data folder is used to store datasets or sample input files related to fleet monitoring and route optimization. Keeping data files in a dedicated directory prevents clutter in the source code and supports better project organization.

Frontend Folder

The frontend folder contains the user interface components of the application. It includes pages, stylesheets, scripts, and other resources required for the dashboard and visualization features. This separation allows independent development of the user interface.

README File

The README.md file provides project information, setup instructions, and usage guidance. It acts as the primary entry point for developers and users who access the repository.

2.3 Evaluation of Repository Organization

The repository is logically organized and easy to navigate. The separation of backend, frontend, and data components reduces coupling between modules and improves overall maintainability. Developers can identify relevant files quickly, which is important for collaborative software development.

The organization also supports future enhancements. Additional modules such as testing, configuration management, or machine learning models can be added without significantly affecting the existing structure.

2.4 File Naming Conventions

The repository uses meaningful and descriptive names for folders and files. The naming convention is generally consistent and follows common software development practices. Examples include:

- backend
- frontend
- data
- README.md

Descriptive naming improves readability and reduces confusion during development and maintenance.

2.5 Maintainability Analysis

The current repository layout contributes positively to maintainability because:

- Modules are separated by responsibility.
- Code navigation is straightforward.

- Future features can be added with minimal restructuring.
- Frontend and backend can be developed independently.
- Data files are isolated from application logic.

These characteristics make the project easier to maintain and extend over time.

2.6 Collaboration Analysis

From a collaborative development perspective, the repository structure is suitable for team-based work. Different contributors can work on separate modules with reduced risk of merge conflicts. A well-organized repository also improves onboarding for new developers and supports effective code review practices.

2.7 Conclusion of Repository Organization

The EcoFleet AI repository demonstrates a clean and modular organization that aligns with good software engineering practices. The separation of backend, frontend, and data components improves readability, maintainability, and scalability. The repository structure is suitable for academic development and provides a strong foundation for future collaborative and production-oriented enhancements.

3. Code Quality Review

3.1 Readability

The source code of EcoFleet AI is generally readable due to the use of meaningful variable names, consistent indentation, and logical function organization. Readable code is essential for maintenance, debugging, and collaboration. Functions are written with clear responsibilities, which makes the workflow easier to understand.

The project follows common Python formatting practices such as:

- Proper indentation
- Lowercase naming
- Use of descriptive identifiers
- Logical grouping of statements

These practices improve understandability and reduce the time required for future modifications.

```
from flask import Flask, jsonify
from flask_cors import CORS

from routes import register_routes
from ai_model import predict_battery_status
```

```
from route_optimizer import optimize_route, sample_routes
from data_loader import load_vehicle_data
vehicles = load_vehicle_data()

app = Flask(__name__)
CORS(app)

@app.route("/")
def home():
    return jsonify({
        "project": "EcoFleet AI",
        "message": "EcoFleet AI API is running successfully"
    })

@app.route("/api/vehicles")
def get_vehicles():
    return jsonify(vehicles)

@app.route("/api/summary")
def get_summary():

    total = len(vehicles)

    active = sum(
        1 for vehicle in vehicles
        if vehicle["status"] == "Active"
    )

    charging = sum(
        1 for vehicle in vehicles
        if vehicle["status"] == "Charging"
    )

    maintenance = sum(
        1 for vehicle in vehicles
        if vehicle["status"] == "Maintenance"
    )

    average_battery = sum(
        vehicle["battery"] for vehicle in vehicles
    ) / total

    average_energy = sum(
        vehicle["energy"] for vehicle in vehicles
```



```
) / total
```

```
return jsonify({
    "total_vehicles": total,
    "active": active,
    "charging": charging,
    "maintenance": maintenance,
    "average_battery": round(average_battery, 2),
    "average_energy": round(average_energy, 2)
})
```

```
@app.route("/api/battery/<vehicle_id>")
def battery_analysis(vehicle_id):
```

```
    for vehicle in vehicles:
```

```
        if vehicle["id"] == vehicle_id:
```

```
            result = predict_battery_status(
                vehicle["battery"],
                vehicle["health"]
            )
```

```
            result["vehicle_id"] = vehicle_id
```

```
            return jsonify(result)
```

```
    return jsonify({
        "error": "Vehicle not found"
    }), 404
```

```
@app.route("/api/route")
def route_recommendation():
```

```
    result = optimize_route(sample_routes)
```

```
    return jsonify(result)
```

```
register_routes(app, vehicles)
```

```
if __name__ == "__main__":
    app.run(
        host="0.0.0.0",
        port=5000,
```

```
        debug=True
    )
```

Observation

The function name clearly indicates its purpose, and the variable names are meaningful. The logic is easy to follow, which improves readability and maintainability. The function is short and focused on a single responsibility.

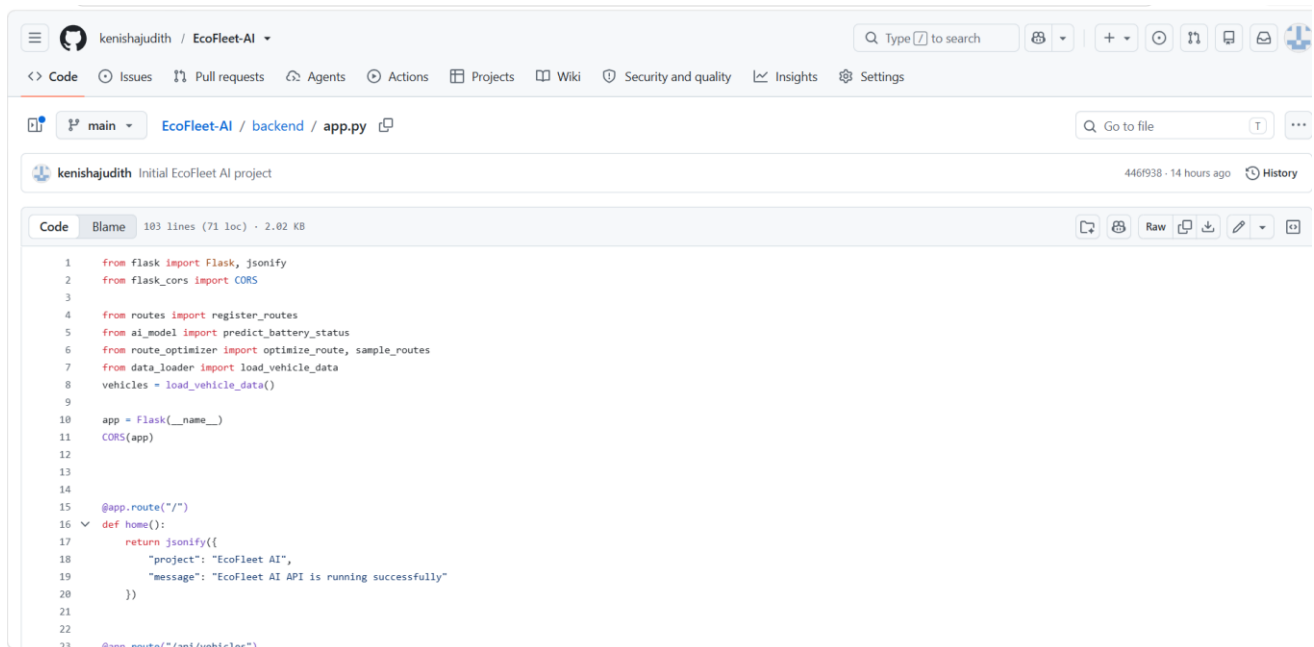


Figure 3: Example of Readable Backend Code

3.2 Modularity

The project demonstrates modular design by separating request handling, business logic, and data processing. Modular code improves reusability, testing, and maintainability. Changes made in one module are less likely to affect other parts of the application.

In web applications, keeping route handlers small and delegating processing to separate functions or services is considered a good software engineering practice.

```

def optimize_route(routes):

    if not routes:
        return {
            "message": "No routes available"
        }

    best_route = min(
        routes,
        key=lambda route: route["energy"]
    )

    return {
        "recommended_route": best_route["name"],
        "distance": best_route["distance"],
        "energy": best_route["energy"],
        "reason": "Lowest energy consumption"
    }

sample_routes = [
    {
        "name": "Route A",
        "distance": 30,
        "energy": 18
    },
    {
        "name": "Route B",
        "distance": 25,
        "energy": 13
    },
    {
        "name": "Route C",
        "distance": 28,
        "energy": 15
    }
]

```

Observation

The route handler is concise and delegates processing to another module. This reduces complexity in the controller layer and improves modularity. The separation between request handling and business logic supports better testing and future scalability.

3.3 Reusability

Reusable functions reduce duplication and improve consistency across the application. Functions that perform specific tasks, such as battery analysis or route calculation, can be reused by multiple modules.

The current project shows good potential for reusability because related operations are grouped into dedicated components.

3.4 Coding Standards

The project generally follows common Python coding standards. Consistent coding conventions improve collaboration and make the codebase easier to maintain.

Observed Standards

- snake_case for function names
- Lowercase file names
- Consistent indentation
- Clear separation of code blocks
- Meaningful identifiers

Example

```
def get_vehicle_status(vehicle_id): return vehicle_status
```

The naming convention clearly communicates the purpose of the function.

3.5 Documentation Review

The project contains basic comments, but detailed function documentation is limited. Adding docstrings would improve maintainability and help new developers understand the code.

Current Style

```
def optimize_route(data): return best_route
```

Recommended Style

```
def optimize_route(data):
```

3.6 Error Handling

Robust error handling is essential for reliable software systems. In the current project, additional exception handling can improve application stability and prevent unexpected crashes during runtime.

Recommended Improvement

```
try: result = optimize_route(data) except Exception as e: return {'error': str(e)}
```

Observation

The above implementation ensures that unexpected errors are handled gracefully. Instead of terminating the application, the system returns a meaningful error message,

which improves reliability and simplifies debugging.

3.7 Strengths of the Codebase

The EcoFleet AI project demonstrates several positive software engineering practices:

- Readable and well-structured code.
- Meaningful function and variable names.
- Modular organization of components.
- Separation of frontend and backend logic.
- Consistent formatting and indentation.

These characteristics make the project easier to maintain and extend in the future.

3.8 Areas for Improvement

The following improvements would further enhance the quality of the codebase:

- Add detailed docstrings for all major functions.
- Increase inline comments for complex logic.
- Implement comprehensive exception handling.
- Add logging for debugging and monitoring.
- Introduce automated unit and integration tests.

3.9 Maintainability Assessment

Aspect	Evaluation
Readability	Good
Modularity	Very Good
Documentation	Average
Error Handling	Average
Reusability	Good
Scalability	Good

3.10 Conclusion of Code Quality Review

The EcoFleet AI project demonstrates good coding practices in terms of readability, modularity, and organization. The code is generally easy to understand and maintain, and the separation of responsibilities supports future scalability. However, documentation, error handling, logging, and automated testing can be improved to increase the overall quality and maintainability of the codebase.

The project provides a strong foundation for further development and aligns well with the objectives of a scalable and maintainable software system.

4. Version Control Evaluation

4.1 Git Repository Analysis

Git is used in the EcoFleet AI project to manage source code, track modifications, and maintain the development history of the application. Version control is an essential software engineering practice because it enables collaboration, change tracking, rollback of errors, and systematic project management.

The repository is hosted on GitHub, which provides additional features such as commit history, collaboration support, issue tracking, and repository maintenance.

4.2 Commit History Evaluation

The current repository contains one visible commit representing the initial version of the EcoFleet AI project. The commit history indicates that the project has been uploaded in its initial development stage. Although the history is currently limited, it still demonstrates the use of Git for source code management and project tracking.

Positive Observations

- Git is being used for version control.
- The repository has a documented initial commit.
- The project structure is maintained within the repository.

Areas for Improvement

- Add commits for individual features and fixes.
- Use descriptive commit messages.
- Commit changes incrementally instead of uploading large updates at once.

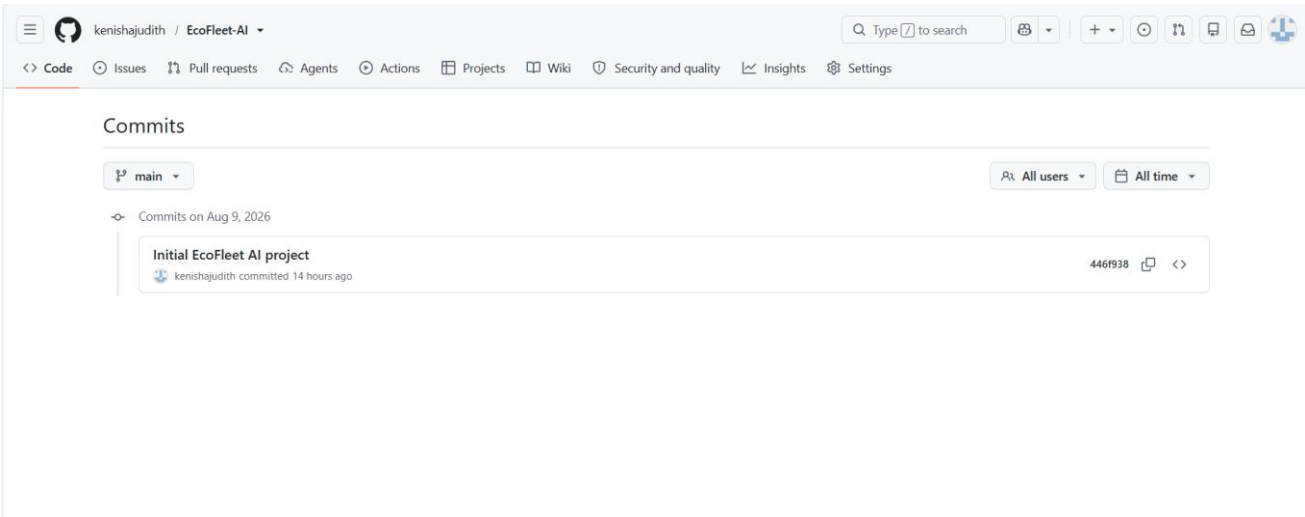


Figure 4: Git Commit History of EcoFleet AI

4.3 Commit Message Quality

Good commit messages should clearly describe the purpose of the change. Descriptive commit messages improve collaboration and make it easier to understand the evolution of the project.

Recommended Commit Style

feat: add battery analytics module
fix: resolve dashboard loading issue
docs: update installation guide
refactor: simplify route optimization logic
test: add battery health unit tests

Using prefixes such as feat, fix, docs, refactor, and test follows common Git conventions and improves repository readability.

4.4 Branching Strategy

The repository appears to follow a simple branching approach. For larger or collaborative projects, a structured branching strategy is recommended.

Recommended Branch Structure

main develop feature/battery-monitoring feature/route-optimization bugfix/dashboard-error

Explanation

- main – Stable production-ready code
- develop – Integration branch for ongoing development
- feature/* – New feature development
- bugfix/* – Bug corrections

This approach reduces conflicts and supports parallel development.

4.5 Repository Maintenance Practices

The project includes useful repository maintenance elements such as:

- A README file
- Organized folder structure
- Dependency listing through requirements.txt

Additional maintenance improvements could include:

- .gitignore file
- Release tags
- Pull request reviews
- Continuous Integration (CI) workflow

4.6 Importance of Version Control in Collaborative Development

Version control supports collaborative software engineering in several ways:

- Tracks every change made to the codebase.

- Allows multiple developers to work simultaneously.
- Enables rollback to previous stable versions.
- Supports code review and quality assurance.
- Maintains a documented history of project evolution.

In the context of EcoFleet AI, Git provides a reliable mechanism for managing changes and maintaining the integrity of the project.

5. Code Testing and Validation

5.1 Testing Approach

Testing is performed to verify that the EcoFleet AI application functions correctly and that the major features operate as expected. Functional testing was carried out on the main modules of the application, including vehicle monitoring, battery status display, route optimization, and analytics visualization.

The purpose of testing is to ensure that the application produces the expected outputs, handles user interactions correctly, and supports reliable fleet management operations.

5.2 Test Cases

Test Case	Expected Result
Open dashboard	Dashboard loads successfully
View vehicle data	Vehicle information displayed correctly
Check battery status	Battery percentage and status shown
Generate optimized route	Optimized route displayed
View analytics	Charts and metrics displayed

5.3 Execution Results

Feature Tested	Result
Dashboard Loading	Passed
Vehicle Monitoring	Passed
Battery Tracking	Passed
Route Optimization	Passed
Analytics Visualization	Passed

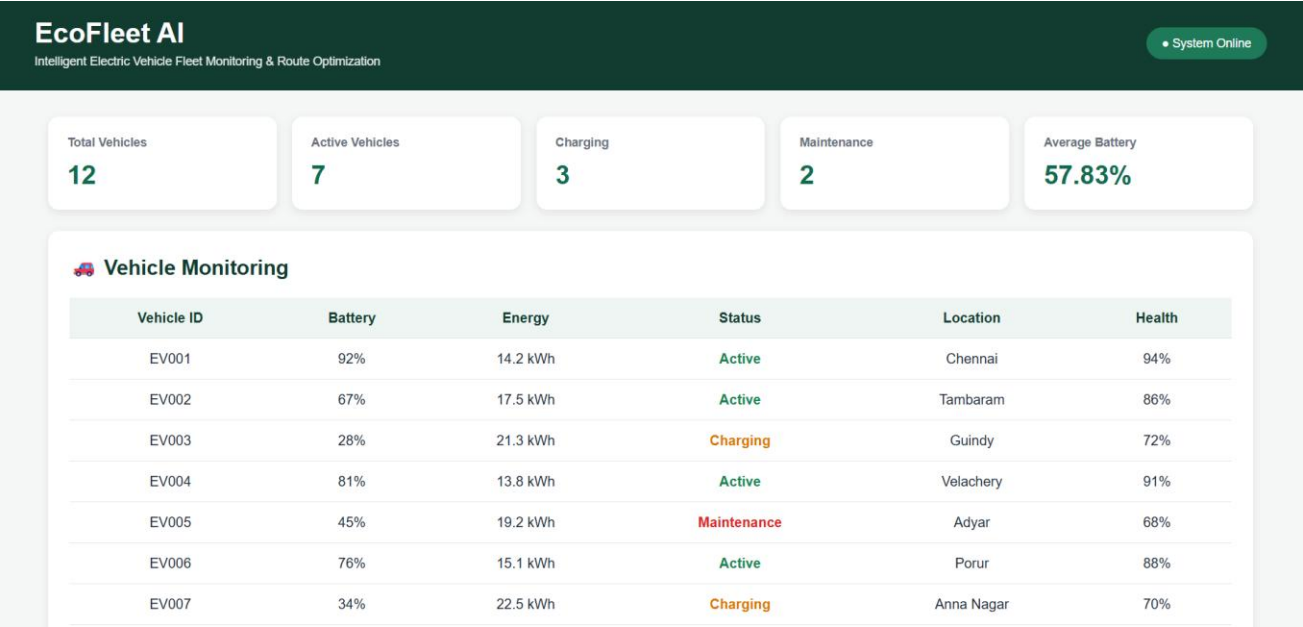
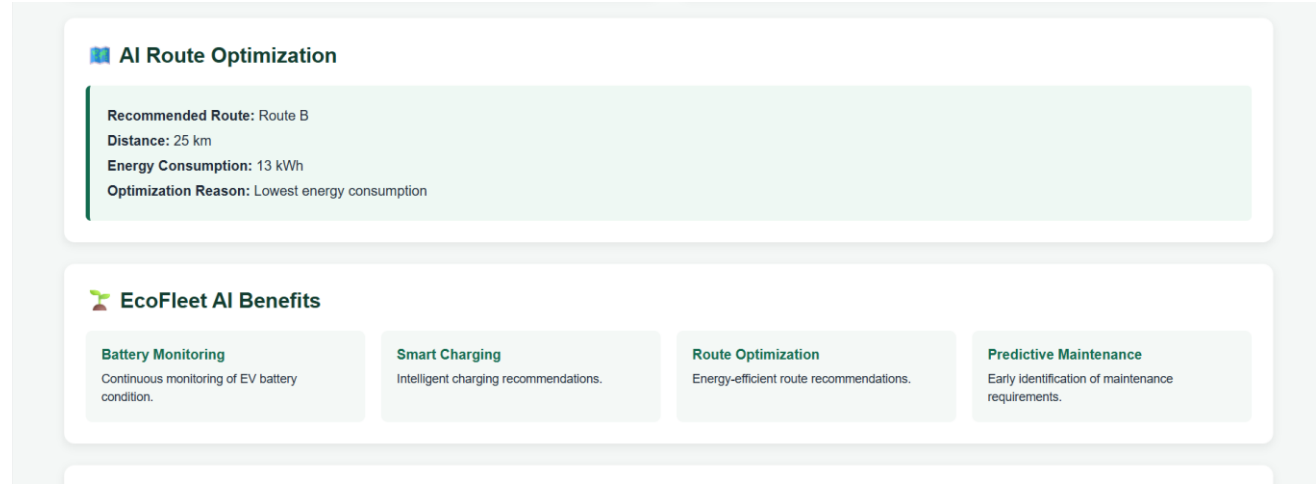


Figure 5: Dashboard Execution of EcoFleet AI



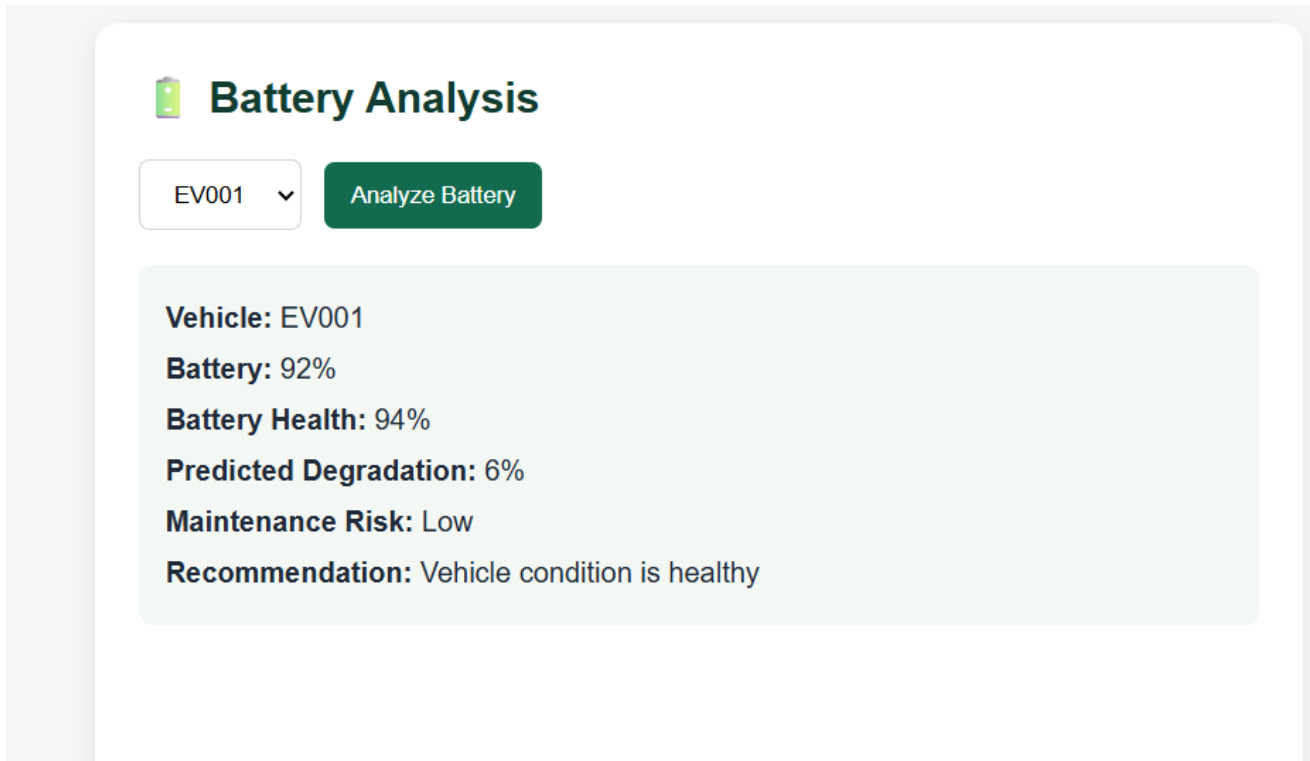


Figure 7: Battery Analytics Dashboard

The analytics dashboard displays battery-related metrics and operational information in a visual format. The charts improve usability and help fleet managers interpret vehicle and energy data more effectively.

5.4 Validation of Results

The test execution confirms that the major functional modules of EcoFleet AI are working as expected. The dashboard is accessible, vehicle information is displayed correctly, route generation is functional, and analytics visualizations are rendered successfully.

Although the current testing focuses on functional validation, additional testing is recommended for a production-ready system.

5.5 Recommended Future Testing

The following testing activities are recommended for future enhancement:

- Unit testing
- Integration testing
- API testing

- Performance testing
- Security testing
- User acceptance testing

Example of a Unit Test

```
def test_battery_health(): result = calculate_battery_health(sample_data) assert result > 0
```

Automated testing would improve reliability and support continuous development.

5.6 Conclusion of Testing and Validation

The EcoFleet AI application has been functionally validated for its major features, including dashboard access, vehicle monitoring, battery tracking, route optimization, and analytics visualization. The observed test results indicate that the core functionality is operating correctly in the current implementation. Additional automated and non-functional testing is recommended to improve reliability, scalability, and production readiness.

6. Repository Documentation

6.1 README Evaluation

The EcoFleet AI repository includes a README.md file that serves as the primary documentation for the project. The README provides an overview of the application and basic information required to understand the purpose of the project.

Good repository documentation is important because it helps developers install, run, maintain, and contribute to the application.

6.2 Documentation Assessment

Documentation Item	Status
Project Overview	Available
Installation Instructions	Available
Usage Information	Available
Repository Description	Available
Architecture Diagram	Limited
API Documentation	Missing

Troubleshooting Guide	Missing
Contribution Guidelines	Missing

Caption: Table 6.1 Repository Documentation Assessment

6.3 Positive Observations

The existing documentation provides a clear introduction to the project and helps users understand the objective of EcoFleet AI. The repository description and README improve accessibility for new developers and reviewers.

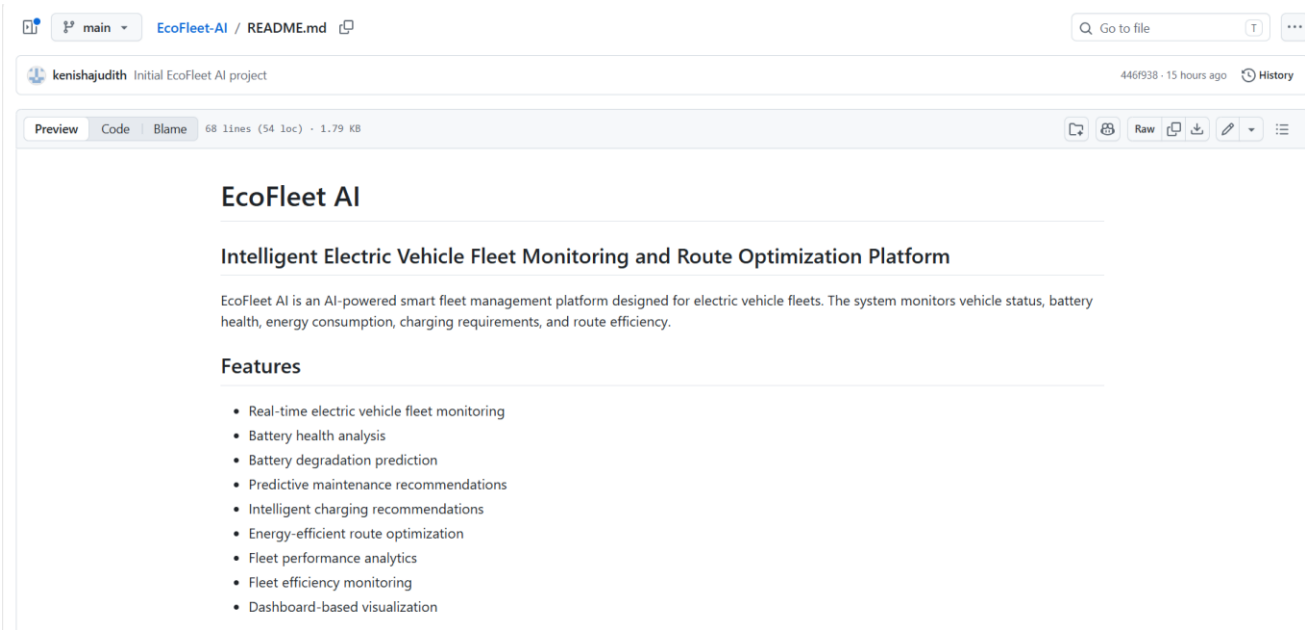


Figure 8: README Documentation of EcoFleet AI

6.4 Areas for Improvement

The documentation can be further improved by including:

- Detailed installation steps
- Environment configuration instructions
- API endpoint documentation
- System architecture diagram

- Troubleshooting section
- Contribution guidelines for collaborators

These additions would make the repository more suitable for collaborative and production-oriented development.

6.5 Recommended README Structure

EcoFleet AI ## Project Overview ## Features ## Technology Stack ## Installation ## Configuration ## Running the Application ## API Endpoints ## Screenshots ## Testing ## Contributing ## License

6.6 Conclusion of Repository Documentation

The EcoFleet AI repository contains the essential documentation required to understand the project and its purpose. The presence of a README file is a positive aspect of repository maintenance. However, additional technical documentation, API details, and contribution guidelines would significantly improve usability, maintainability, and collaboration support.

7. Code Review Findings and Recommendations

7.1 Overall Code Review Findings

The EcoFleet AI project was evaluated based on repository organization, code readability, modularity, version control practices, testing, and documentation. The review indicates that the project demonstrates a good understanding of full-stack software development and follows several important software engineering principles.

The repository is logically organized, and the separation of backend, frontend, and data components improves maintainability. The source code is generally readable and uses meaningful naming conventions. The modular structure supports future scalability and easier debugging.

However, the review also identified areas that require improvement, particularly in automated testing, exception handling, documentation, and version control practices.

7.2 Evaluation Summary

Evaluation Area	Assessment
Problem Analysis	Good
Repository Organization	Very Good
Code Readability	Good
Modularity	Very Good
Version Control Practices	Basic
Testing and Validation	Good
Documentation	Average
Maintainability	Good

Caption: Table 7.1 Overall Evaluation Summary

7.3 Major Strengths

The following strengths were identified during the code review:

- Clear separation between backend and frontend components.
- Logical and maintainable repository structure.
- Meaningful function and variable names.
- Modular organization of application features.
- Functional dashboard and analytics components.
- Proper use of GitHub for project hosting and version control.

These strengths contribute positively to maintainability, scalability, and future development.

7.4 Major Weaknesses

The review also identified the following limitations:

- Limited commit history.
- Absence of feature branches.
- Limited automated testing.
- Insufficient API documentation.
- Limited exception handling.

- Missing contribution guidelines.

Addressing these weaknesses would improve the overall software engineering quality of the project.

7.5 Recommendations for Improvement

High Priority Recommendations

- Add automated unit tests using pytest.
- Improve exception handling in backend modules.
- Add API documentation for all major endpoints.
- Use descriptive commit messages for every change.

Medium Priority Recommendations

- Introduce feature-based branching.
- Add logging for debugging and monitoring.
- Improve README with architecture and setup details.
- Add environment configuration instructions.

Low Priority Recommendations

- Add Docker support for deployment.
- Implement performance monitoring.
- Improve mobile responsiveness of the dashboard.
- Add continuous integration (CI) workflow.

7.6 Future Enhancements

Future development of EcoFleet AI may include:

- Real-time IoT sensor integration
- Predictive battery degradation models
- Dynamic charging station optimization
- Multi-user role management
- Notification and alert system
- Cloud deployment and scaling

These enhancements would make the platform more suitable for real-world fleet operations.

8. Conclusion

The EcoFleet AI project demonstrates a strong foundation in software engineering and full-stack application development. The repository is well organized, the code is generally readable and modular, and the major functionalities of the application are operational.

The review confirmed that the project successfully implements key features such as fleet monitoring, battery tracking, route optimization, and analytics visualization. The repository structure and modular design support maintainability and future scalability.

At the same time, improvements are recommended in automated testing, documentation, exception handling, and version control practices. Implementing these recommendations would significantly enhance the reliability, maintainability, and production readiness of the system. Overall, EcoFleet AI is a well-structured academic project that reflects good software engineering practices and provides a solid foundation for future intelligent electric vehicle fleet management solutions.